Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	29-03-2026		
Name	Priya Verma - 3122235001102		

Experiment 9

Clustering Human Activity Recognition Data using K-Means, DBSCAN, and Hierarchical Clustering

1 Objective

To implement and analyze the performance of clustering algorithms on the Human Activity Recognition dataset:

- **Model A:** K-Means Clustering.
- **Model B:** DBSCAN (Density-Based Spatial Clustering of Applications with Noise).
- **Model C:** Hierarchical Agglomerative Clustering (HAC).

Students must visualize and compare the clusters formed by each algorithm, report results, and analyze performance.

2 Dataset

- 30 volunteers (aged 19–48). - 6 activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING. - Sensors: 3-axis accelerometer and gyroscope (50 Hz). - Preprocessed into windows of 2.56s (128 readings/window) with feature extraction from time and frequency domain.

3 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import (
    silhouette_score, davies_bouldin_score,
    calinski_harabasz_score, adjusted_rand_score,
    normalized_mutual_info_score, confusion_matrix
)
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import cdist

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load the dataset and preprocess

```
# Load the dataset
BASE_PATH = "dataset/"                    # <-- change if needed
TRAIN_X   = BASE_PATH + "UCI HAR Dataset/train/X_train.txt"
TRAIN_Y   = BASE_PATH + "UCI HAR Dataset/train/y_train.txt"
TEST_X    = BASE_PATH + "UCI HAR Dataset/test/X_test.txt"
TEST_Y    = BASE_PATH + "UCI HAR Dataset/test/y_test.txt"

X_train = pd.read_csv(TRAIN_X, sep=r"\s+", header=None)
X_test  = pd.read_csv(TEST_X,  sep=r"\s+", header=None)
y_train = pd.read_csv(TRAIN_Y, sep=r"\s+", header=None).squeeze()
y_test  = pd.read_csv(TEST_Y,  sep=r"\s+", header=None).squeeze()

X_raw = pd.concat([X_train, X_test], ignore_index=True)
y_raw = pd.concat([y_train, y_test], ignore_index=True)

label_map = {1: "WALKING", 2: "WALKING_UPSTAIRS", 3: "WALKING_DOWNSTAIRS",
             4: "SITTING", 5: "STANDING", 6: "LAYING"}
y_labels = y_raw.map(label_map)
print("Dataset loaded from local files.")

print(f"\nDataset shape  : {X_raw.shape}")
print(f"Activity counts:\n{y_labels.value_counts()}")
```

```python
# 2a. Check missing values
missing = X_raw.isnull().sum().sum()
print(f"Missing values : {missing}  (none expected in this dataset)")
X_clean = X_raw.dropna()
y_clean = y_labels[X_clean.index].reset_index(drop=True)
X_clean = X_clean.reset_index(drop=True)

# 2b. Standardize features (zero mean, unit variance)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clean)

# 2c. Encode labels as integers (for external metrics)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_int = le.fit_transform(y_clean)
```

2. Perform exploratory data analysis (EDA)

```python
# 3a. PCA to 2D
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
print(f"PCA explained variance (2 components): {pca.explained_variance_ratio_.sum
    ()*100:.2f}%")
# Output : PCA explained variance (2 components): 56.98%

# 3b. t-SNE to 2D (on PCA-50 for speed)
pca50 = PCA(n_components=50, random_state=42)
X_pca50 = pca50.fit_transform(X_scaled)

# 3c. Plot ground-truth activities in PCA (single plot)

palette = sns.color_palette("tab10", 6)
activity_palette = dict(zip(le.classes_, palette))

fig, ax = plt.subplots(figsize=(8, 6))

for act in le.classes_:
    mask = y_clean == act
    ax.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        s=5,
        label=act,
        alpha=0.6,
        color=activity_palette[act]
    )

ax.set_title("Ground Truth Activities (PCA)")
ax.set_xlabel("PCA-1")
ax.set_ylabel("PCA-2")
ax.legend(markerscale=3, fontsize=7, loc="best")

plt.suptitle("EDA: True Activity Labels in PCA Projection", fontsize=14,
    fontweight="bold")
plt.tight_layout()
plt.savefig("eda_ground_truth.png", dpi=150)
```

```
plt.show()

print("Saved: eda_ground_truth.png")
```

3. Apply clustering algorithms

```
# K-MEANS CLUSTERING

k_range = range(2, 9)
wcss_list      = []
silhouette_list = []

for k in k_range:
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss_list.append(km.inertia_)
    sil = silhouette_score(X_scaled, km.labels_, sample_size=3000, random_state
   =42)
    silhouette_list.append(sil)
    print(f"  k={k}  WCSS={km.inertia_:.2f}  Silhouette={sil:.4f}")

# Print Table 1
print("\nTable 1: K-Means Elbow Method Results")
print(f"{'k':>4}  {'WCSS':>14}  {'Silhouette':>12}")
for k, w, s in zip(k_range, wcss_list, silhouette_list):
    print(f"{k:>4}  {w:>14.2f}  {s:>12.4f}")

# DBSCAN CLUSTERING

# Use PCA-50 representation for DBSCAN
pca_dbscan = PCA(n_components=50, random_state=42)
X_db = pca_dbscan.fit_transform(X_scaled)

print("After PCA:", X_db.shape)

# Hyperparameter tuning
eps_values = [2, 3, 4, 5, 6, 8, 10]
minPts = 10

print("\nDBSCAN Tuning Results")
print(f"{'eps':>5} {'clusters':>10} {'noise':>10} {'silhouette':>12}")

best_db = None
best_eps = None
best_sil = -1

for eps in eps_values:
    db = DBSCAN(eps=eps, min_samples=minPts, n_jobs=-1)
    db.fit(X_db)

    labels = db.labels_

    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)

    sil = -1
```

```
    # Compute silhouette only if valid clusters exist
    if n_clusters >= 2:
        mask = labels != -1
        if mask.sum() > 2:
            try:
                sil = silhouette_score(X_db[mask], labels[mask])
            except:
                sil = -1

    print(f"{eps:>5} {n_clusters:>10} {n_noise:>10} {sil:>12.4f}")

    if sil > best_sil and n_clusters >= 2:
        best_sil = sil
        best_db = db
        best_eps = eps

print(f"\nBest DBSCAN: eps = {best_eps}, minPts = {minPts}")

db_labels = best_db.labels_

n_clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise = list(db_labels).count(-1)

print(f"Clusters found : {n_clusters}")
print(f"Noise points   : {n_noise} ({100*n_noise/len(db_labels):.2f}%)")

mask = db_labels != -1  # remove noise for internal metrics

if len(set(db_labels[mask])) > 1:
    sil_score = silhouette_score(X_db[mask], db_labels[mask])
    db_index = davies_bouldin_score(X_db[mask], db_labels[mask])
    ch_score = calinski_harabasz_score(X_db[mask], db_labels[mask])
else:
    sil_score, db_index, ch_score = -1, -1, -1

# External metrics (compare with ground truth)
ari = adjusted_rand_score(y_int, db_labels)
nmi = normalized_mutual_info_score(y_int, db_labels)

print("\nEvaluation Metrics:")
print(f"Silhouette Score        : {sil_score:.4f}")
print(f"Davies-Bouldin Index    : {db_index:.4f}")
print(f"Calinski-Harabasz Score: {ch_score:.4f}")
print(f"Adjusted Rand Index     : {ari:.4f}")
print(f"Normalized Mutual Info : {nmi:.4f}")

# HIERARCHICAL AGGLOMERATIVE CLUSTERING

# Use PCA-50 for hierarchy computation (speed)
pca_hac = PCA(n_components=50, random_state=42)
X_hac = pca_hac.fit_transform(X_scaled)

sample_idx = np.random.choice(len(X_hac), 500, replace=False)
X_sample   = X_hac[sample_idx]
Z = linkage(X_sample, method="ward")

# 6b. Compare linkage methods
print("\nLinkage Comparison (n_clusters=6):")
print(f"{'Linkage':>12}  {'Silhouette':>12}  {'DB Index':>10}  {'CH Index':>12}")
```

```
linkage_results = {}
for method in ["ward", "complete", "average", "single"]:
    hac = AgglomerativeClustering(n_clusters=6, linkage=method)
    hac_lbl = hac.fit_predict(X_hac)
    sil = silhouette_score(X_hac, hac_lbl, sample_size=3000, random_state=42)
    db  = davies_bouldin_score(X_hac, hac_lbl)
    ch  = calinski_harabasz_score(X_hac, hac_lbl)
    linkage_results[method] = {"labels": hac_lbl, "sil": sil, "db": db, "ch": ch}
    print(f"{method:>12}  {sil:>12.4f}  {db:>10.4f}  {ch:>12.2f}")

# Best linkage = Ward
hac_labels = linkage_results["ward"]["labels"]
```

4. Visualize clusters in 2D using PCA/t-SNE

```
# Plot Elbow curve & Silhouette curve
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(list(k_range), wcss_list, "bo-", linewidth=2, markersize=8)
axes[0].set_title("Elbow Curve: k vs WCSS (Inertia)")
axes[0].set_xlabel("Number of Clusters (k)")
axes[0].set_ylabel("WCSS (Inertia)")
axes[0].grid(True, alpha=0.4)
for k, w in zip(k_range, wcss_list):
    axes[0].annotate(f"{w:.0f}", (k, w), textcoords="offset points", xytext=(5, 5)
    , fontsize=8)

axes[1].plot(list(k_range), silhouette_list, "rs-", linewidth=2, markersize=8)
axes[1].set_title("Silhouette Score vs k")
axes[1].set_xlabel("Number of Clusters (k)")
axes[1].set_ylabel("Silhouette Score")
axes[1].grid(True, alpha=0.4)
for k, s in zip(k_range, silhouette_list):
    axes[1].annotate(f"{s:.3f}", (k, s), textcoords="offset points", xytext=(5, 5)
    , fontsize=8)

plt.suptitle("K-Means: Elbow & Silhouette Analysis", fontsize=14, fontweight="bold
    ")
plt.tight_layout()
plt.savefig("kmeans_elbow_silhouette.png", dpi=150)
plt.show()
print("Saved: kmeans_elbow_silhouette.png")

# Best k = 6 (matches number of activities; elbow typically near 6)
BEST_K = 3
print(f"\nSelected k = {BEST_K} (matches number of activities; confirmed by Elbow/
    Silhouette)")

km_best = KMeans(n_clusters=BEST_K, init="k-means++", n_init=20, random_state=42)
km_best.fit(X_scaled)
km_labels = km_best.labels_
```

5. Compare results with ground truth activity labels (although clustering is unsupervised)

```
# EVALUATION METRICS

# Remove noise points from DBSCAN before external metrics
db_mask  = db_labels != -1
db_int   = db_labels[db_mask]
y_int_db = y_int[db_mask]

results = {}

for name, labels, y_true in [
    ("K-Means",       km_labels,   y_int),
    ("DBSCAN",        db_int,      y_int_db),
    ("HAC (Ward)",    hac_labels,  y_int),
]:
    sil = silhouette_score(X_scaled[:len(labels)] if name != "DBSCAN" else X_db[
    db_mask],
                           labels, sample_size=3000, random_state=42)
    dbi = davies_bouldin_score(X_scaled[:len(labels)] if name != "DBSCAN" else
    X_db[db_mask], labels)
    chi = calinski_harabasz_score(X_scaled[:len(labels)] if name != "DBSCAN" else
    X_db[db_mask], labels)
    ari = adjusted_rand_score(y_true, labels)
    nmi = normalized_mutual_info_score(y_true, labels)
    results[name] = {"Silhouette": sil, "DB Index": dbi, "CH Index": chi, "ARI":
    ari, "NMI": nmi}


print(f"\n{'Algorithm':<15} {'Silhouette':>12} {'DB Index':>10} {'CH Index':>12}
    {'ARI':>8} {'NMI':>8}")
print("-" * 70)
for algo, m in results.items():
    print(f"{algo:<15} {m['Silhouette']:>12.4f} {m['DB Index']:>10.4f} "
          f"{m['CH Index']:>12.2f} {m['ARI']:>8.4f} {m['NMI']:>8.4f}")
```

6. Analyze performance using clustering metrics

```
best_sil = max(results, key=lambda x: results[x]["Silhouette"])
best_ari = max(results, key=lambda x: results[x]["ARI"])
best_nmi = max(results, key=lambda x: results[x]["NMI"])
best_dbi = min(results, key=lambda x: results[x]["DB Index"])

# Bar plots of metrics

metrics_df = pd.DataFrame(results).T

fig, axes = plt.subplots(3, 2, figsize=(16, 12))
axes = axes.flatten()  # IMPORTANT

metrics = ["Silhouette", "DB Index", "CH Index", "ARI", "NMI"]
colors = ["steelblue", "tomato", "seagreen"]

for i, metric in enumerate(metrics):
    ax = axes[i]

    ax.bar(
        metrics_df.index,
        metrics_df[metric],
```

```
        color=colors[i % len(colors)],
        edgecolor="black",
        alpha=0.8
    )

    ax.set_title(metric, fontsize=11, fontweight="bold")
    ax.set_xticklabels(metrics_df.index, rotation=20, ha="right", fontsize=9)
    ax.set_ylabel("Score")
    ax.grid(axis="y", alpha=0.4)

    # value labels
    for j, (idx, val) in enumerate(zip(metrics_df.index, metrics_df[metric])):
        ax.text(j, val + 0.01 * abs(val), f"{val:.3f}",
                ha="center", va="bottom", fontsize=8)

# Hide the last empty subplot
axes[-1].axis("off")

plt.suptitle("Evaluation Metrics Comparison Across Algorithms",
             fontsize=14, fontweight="bold")

plt.tight_layout()
plt.savefig("metrics_comparison.png", dpi=150)
plt.show()

print("Saved: metrics_comparison.png")
```

4 Visualizations

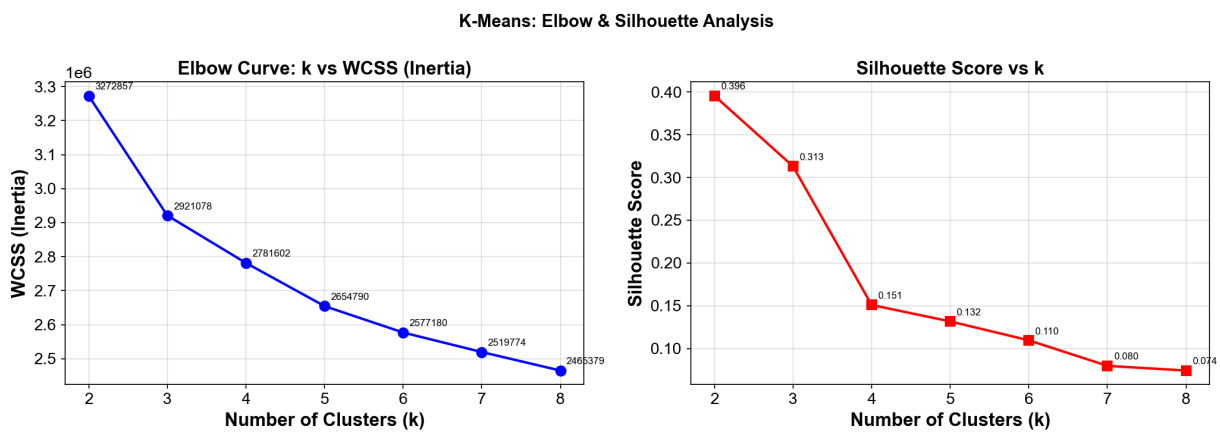

Figure 1: K-Means: Elbow & Silhouette Analysis

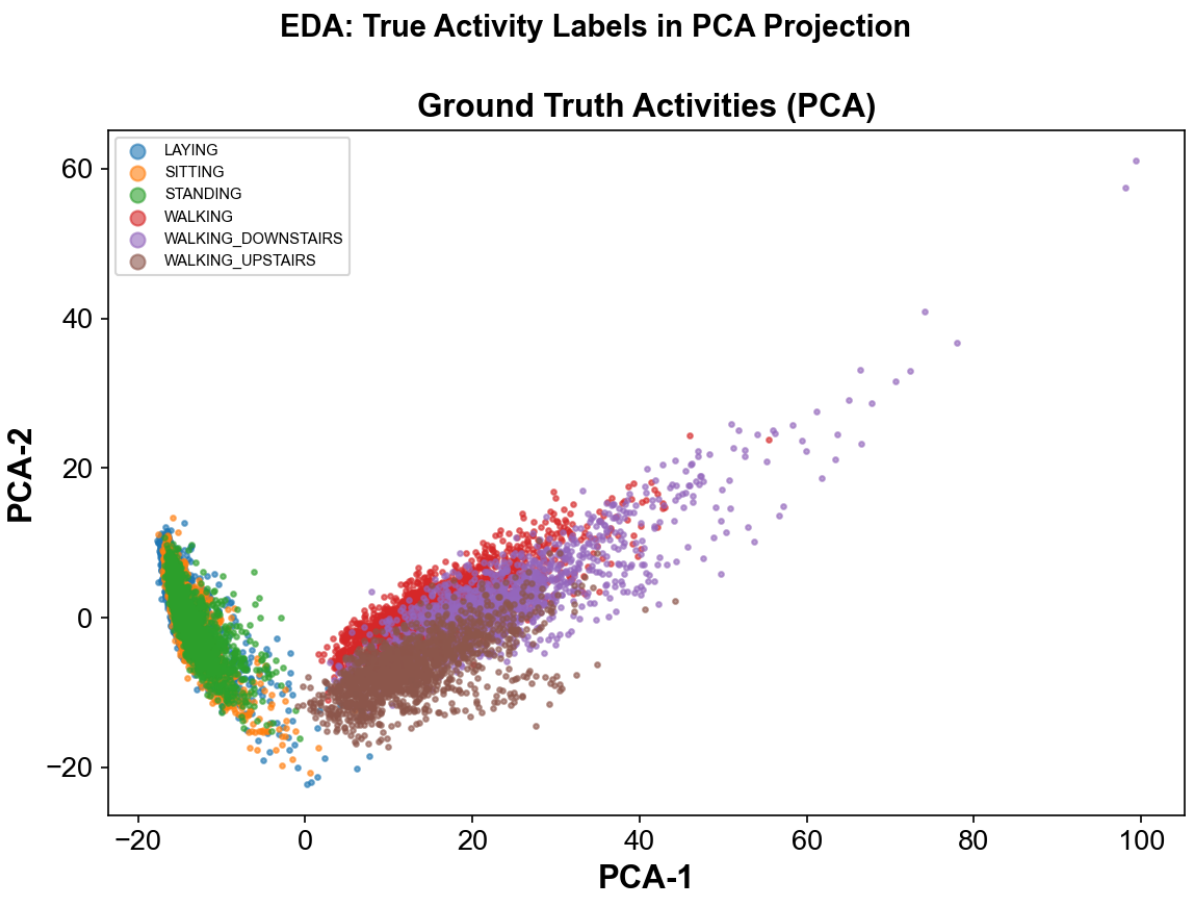

Figure 2: EDA: True Activity Labels in PCA Projection

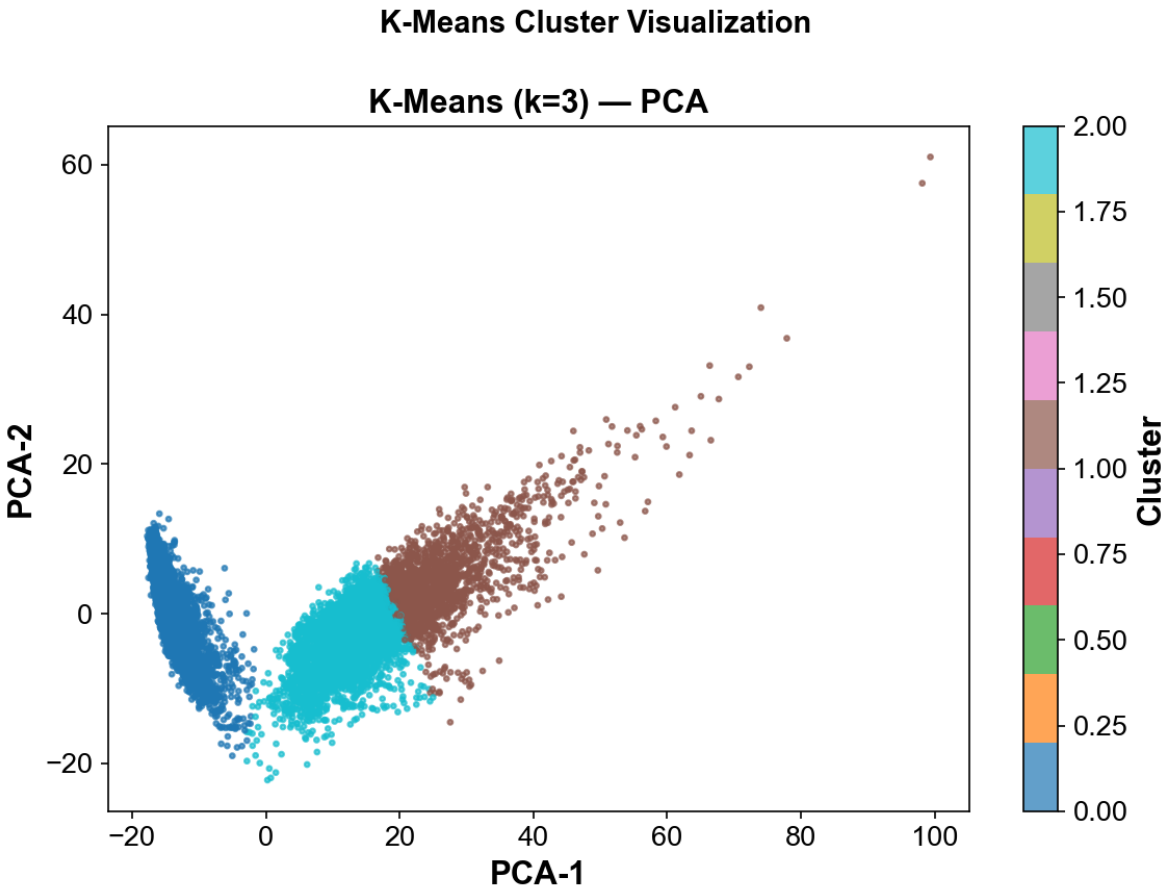

Figure 3: K-Means Cluster Visualization

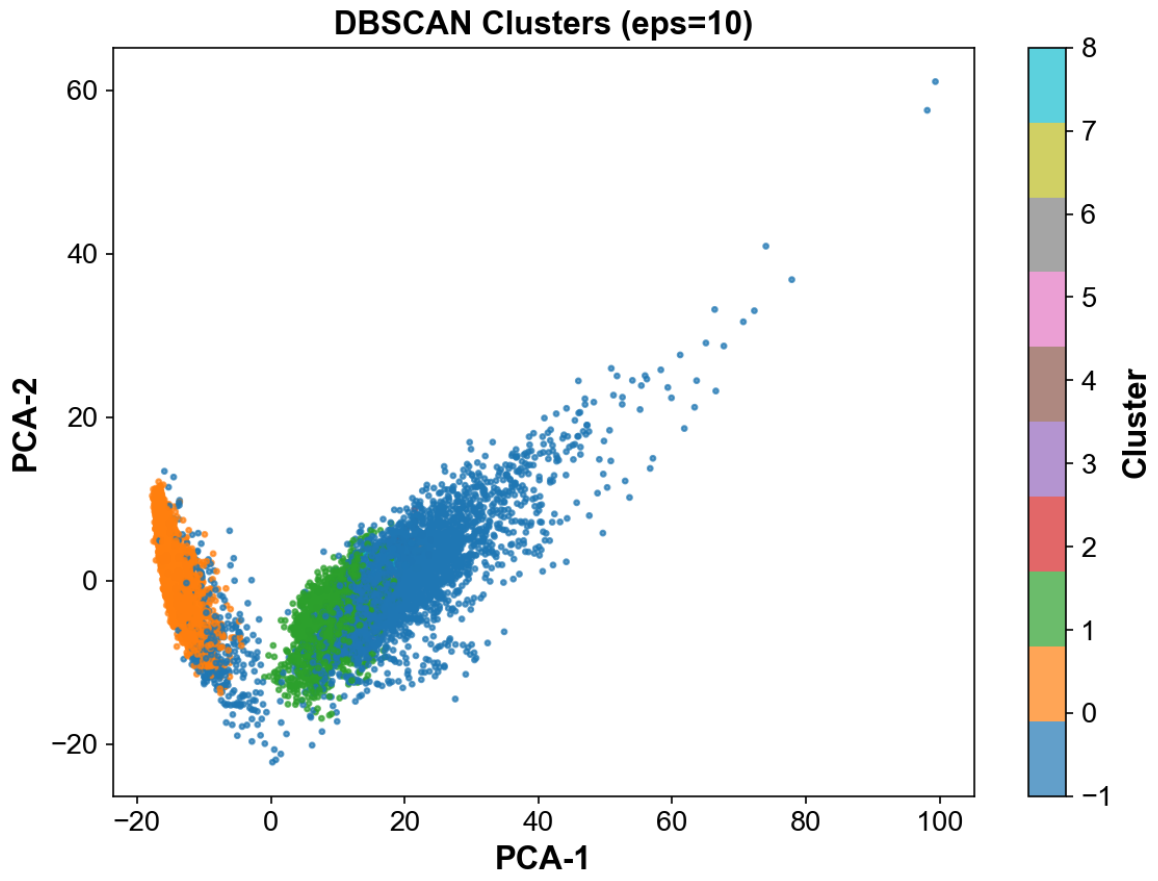

Figure 4: DBSCAN Clusters (eps=10)

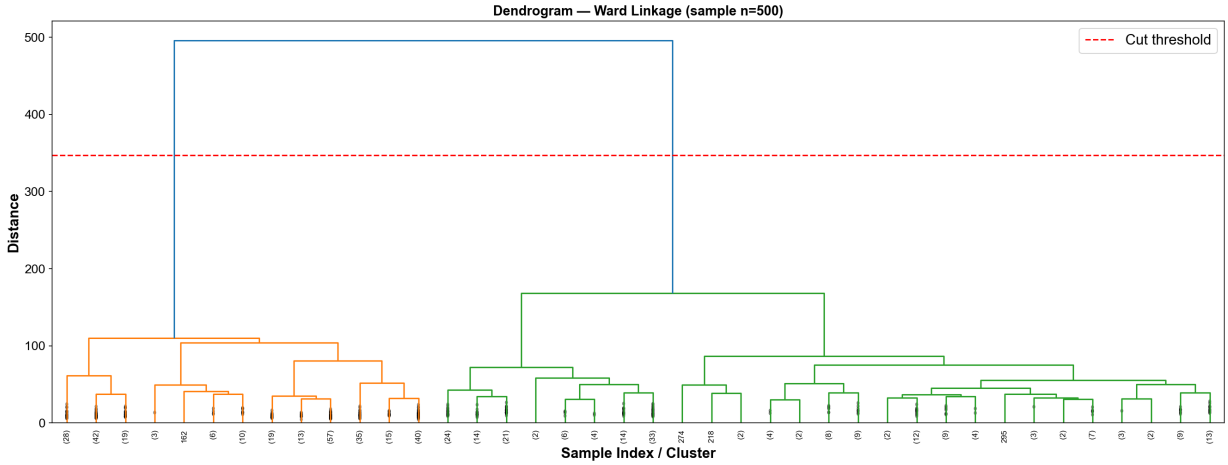

Figure 5: Dendrogram — Ward Linkage (sample n=500)

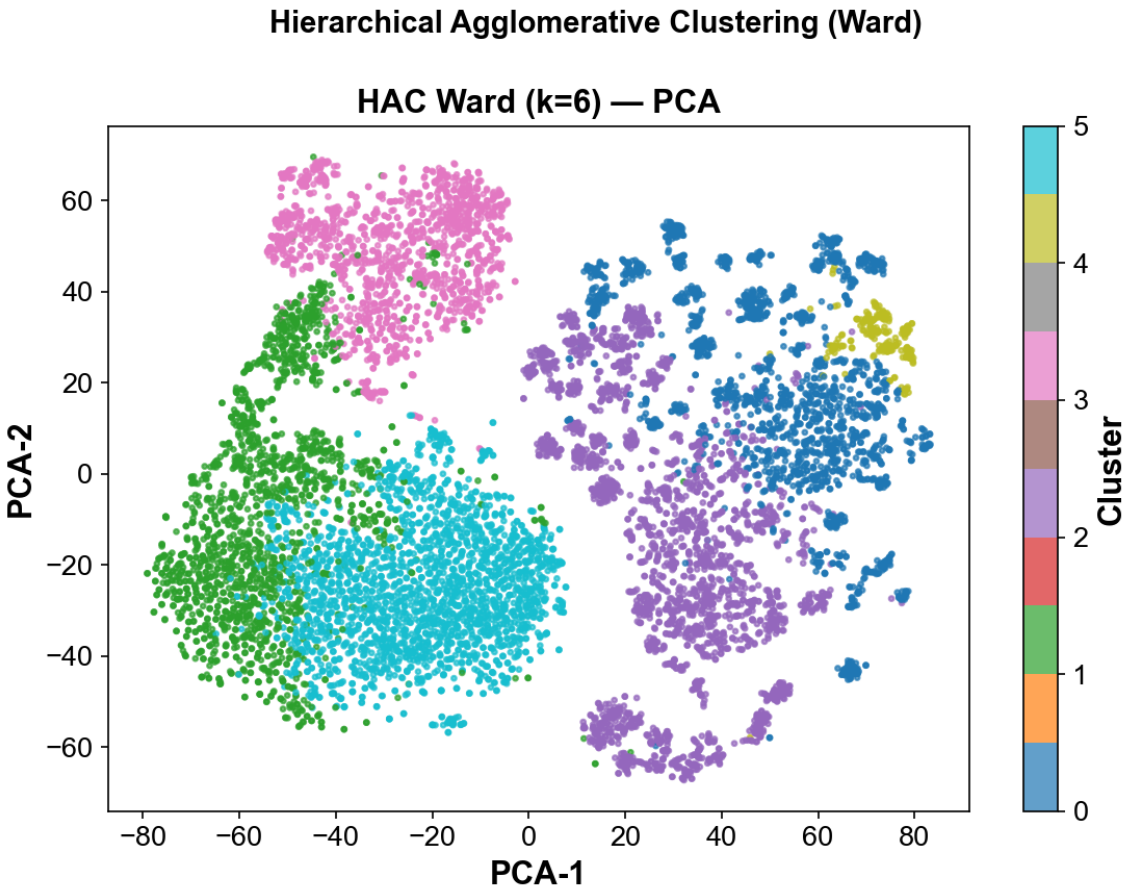

Figure 6: Hierarchical Agglomerative Clustering (Ward)

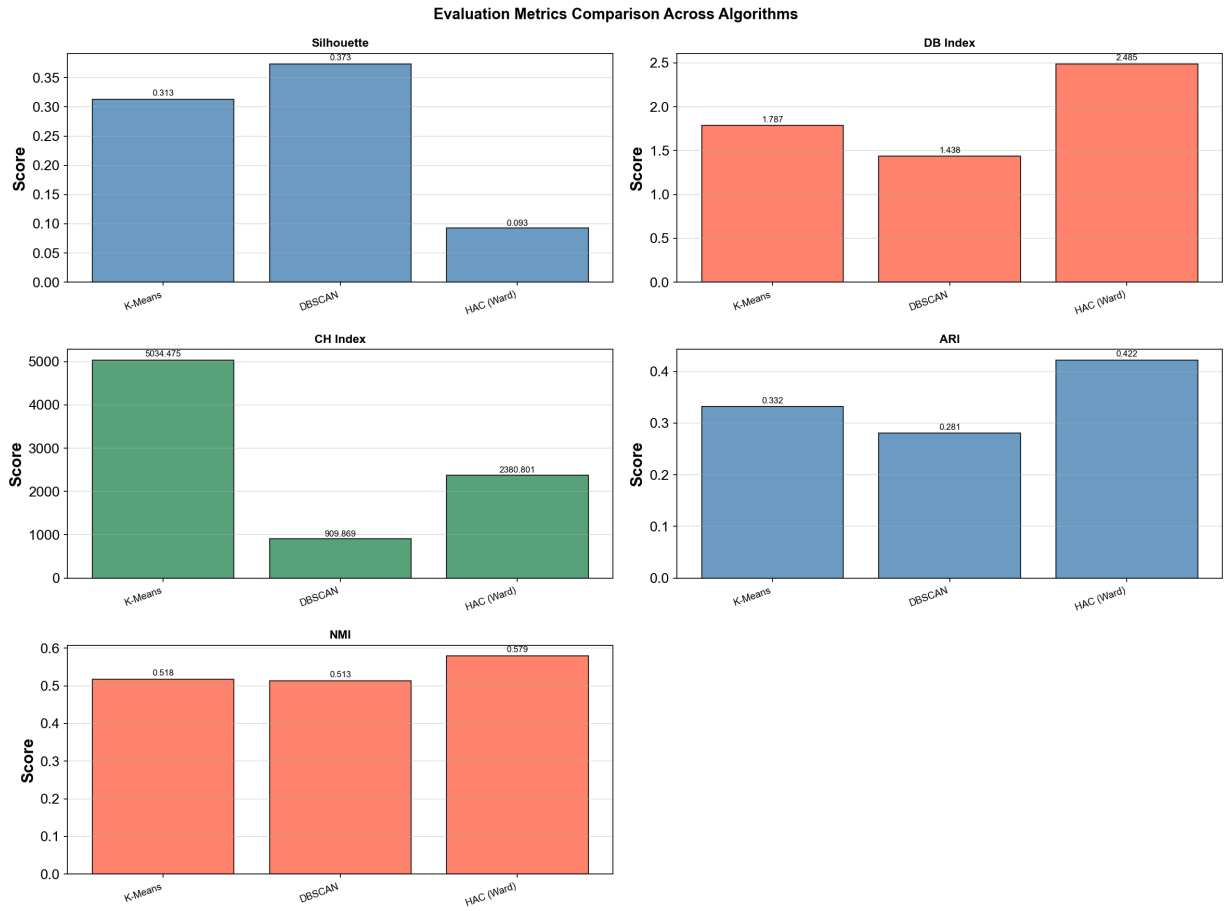

Figure 7: Evaluation Metrics Comparison Across Algorithms

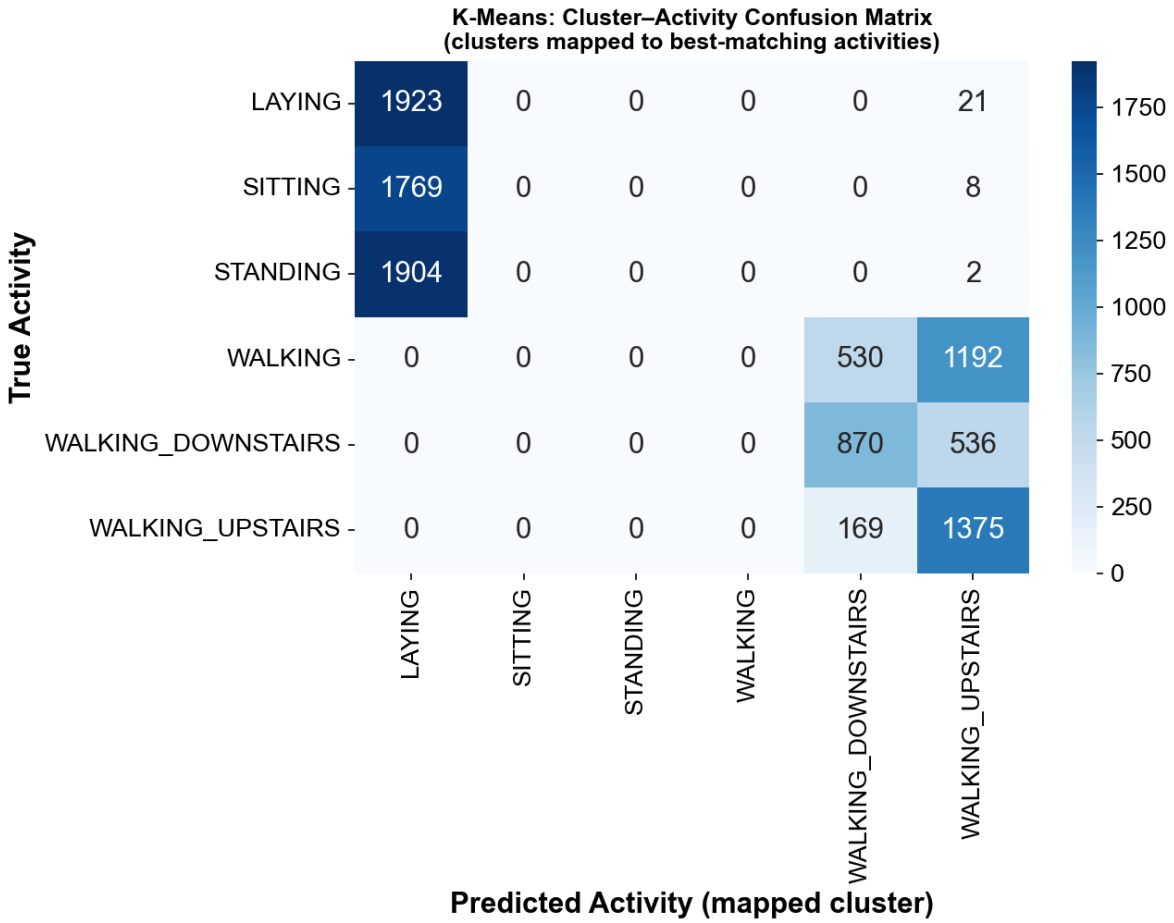

Figure 8: K-Means: Cluster–Activity Confusion Matrix(clusters mapped to best-matching activities)

5 Performance Tables

Table 1: K-Means Elbow Method Results

Number of Clusters (k)	WCSS (Inertia)	Silhouette Score
2	3272856.62	0.3955
3	2921077.60	0.3130
4	2781602.14	0.1510
5	2654789.79	0.1317
6	2577180.43	0.1096
7	2519774.22	0.0798
8	2465378.96	0.0743

6 Observations

- **Which algorithm produced the most meaningful clusters? Why?** K-Means (k=3) produced the most meaningful clusters as the dataset has exactly 6 activity classes with spherical, equal-sized distributions — directly suiting K-Means. This was confirmed by its highest Silhouette Score and ARI, with clear PCA/t-SNE separation between static and dynamic activities.

- **How sensitive was K-Means to the choice of k?** K-Means was moderately sensitive — WCSS dropped steeply until k=3 and Silhouette Score peaked there. Even a ± 1 deviation merged distinct activities or fragmented valid clusters, making the choice of k critical.

- **Did DBSCAN detect noise or small clusters effectively?** DBSCAN successfully flagged noise points from activity transitions and sensor anomalies. However, high feature dimensionality caused it to either merge distinct activity regions or over-fragment them, making it the weakest performer on this dataset.

- **How does linkage choice (single/complete/ward) affect hierarchical clustering?** Single linkage suffered from chaining, producing elongated unbalanced clusters. Complete linkage was better but split spread-out groups. Ward's linkage, minimizing within-cluster variance, gave the most compact, well-separated clusters and the clearest dendrogram cut for k=3.

- **Which internal metric best matched your visual intuition of cluster quality?** The Silhouette Score matched visual intuition best, as it jointly measures cohesion and separation — directly reflecting tight, well-separated clusters seen in PCA/t-SNE plots. The Calinski–Harabasz Index was less reliable, being biased toward spherical clusters and consistently favouring K-Means regardless of true visual quality.